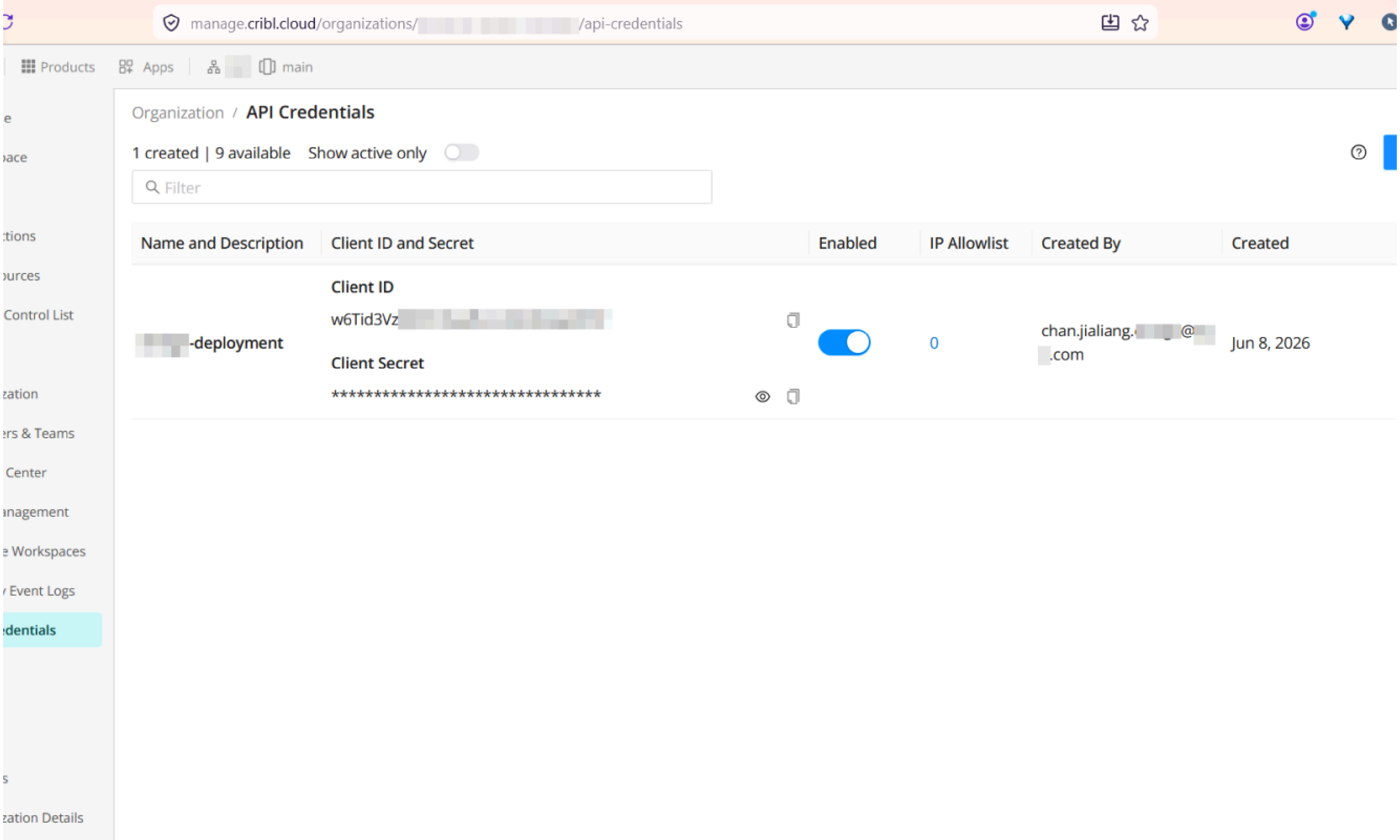
[Cribl] Programmatically Creating Sources via API

Jia Liang Chan
Cyber Security Consultant (Advanced Analytics)

Step 1: Authenticate in Cribl.Cloud and Hybrid Deployments

Refer: https://docs.cribl.io/cribl-as-code/api-auth#api-auth-cloud

In Cribl.Cloud, create a new pair of API credential. The API Credential includes a **Client ID** and a **Client Secret** that Organization Owners and Admins can use to generate Bearer tokens.



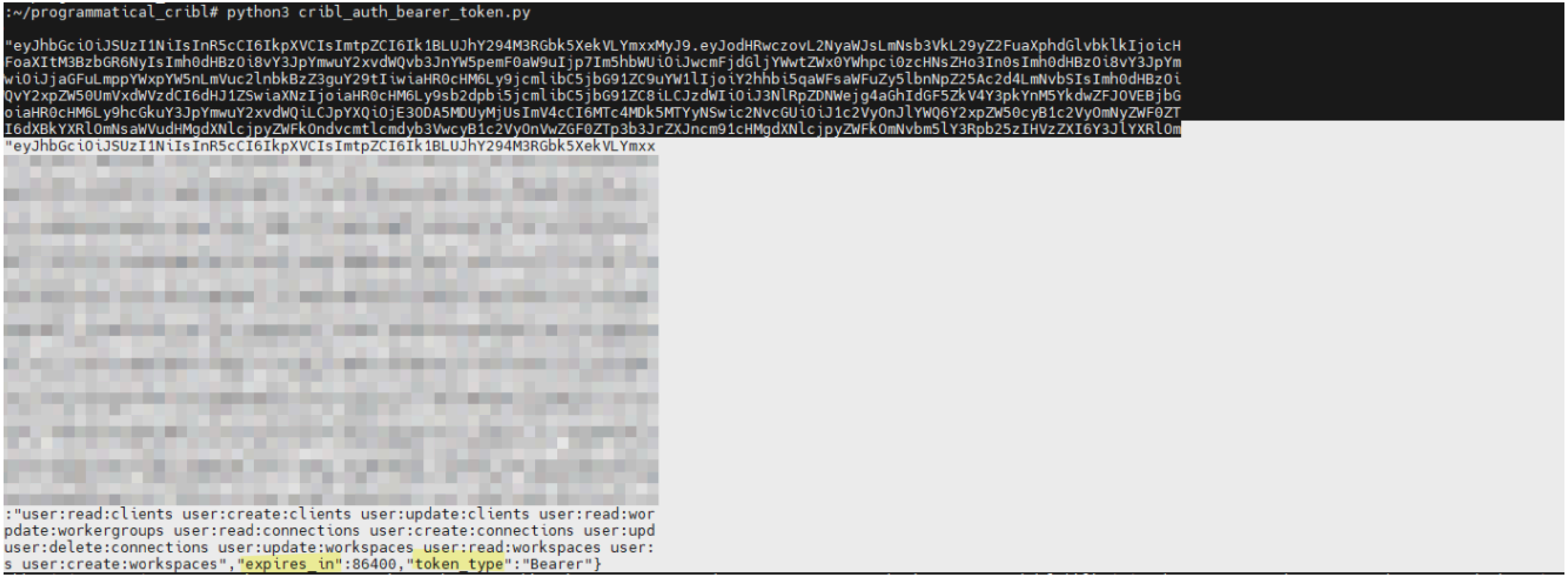
Execute the following cURL command to retrieve an OAuth Bearer token. This token must be included in the Authorization header of subsequent API calls used to create a source.

:hentication Request Example (Cribl.Cloud and Hybrid)

```
curl --request POST \
--url "https://login.cribl.cloud/oauth/token" \
--header "Content-Type: application/json" \
--data "{
  \"grant_type\": \"client_credentials\",
  \"client_id\": \"${clientId}\",
  \"client_secret\": \"${clientSecret}\",
  \"audience\": \"https://api.cribl.cloud\"
}"
```

Cop

The cURL command is executed using a Python-based implementation, with client credentials securely loaded from a `.env` file to ensure sensitive information is not exposed. This step results in the acquisition of an OAuth Bearer token, which is then used to authenticate subsequent API requests.



Step 2: Use `POST /system/inputs` to create a new source

Refer: https://docs.cribl.io/cribl-as-code/api-reference/control-plane/cribl-stream

urces Actions related to Sources

GET	`/system/inputs`	List all Sources
POST	`/system/inputs`	Create a Source
GET	`/system/inputs/{id}`	Get a Source

Using `POST /system/inputs`

POST `/system/inputs` Create a Source

Create a new Source. The system-managed provenance field (JSON `criblSourceProvenance`) must be omitted from the request body.

Parameters

No parameters

Request body required application/json

Input object.

Examples:

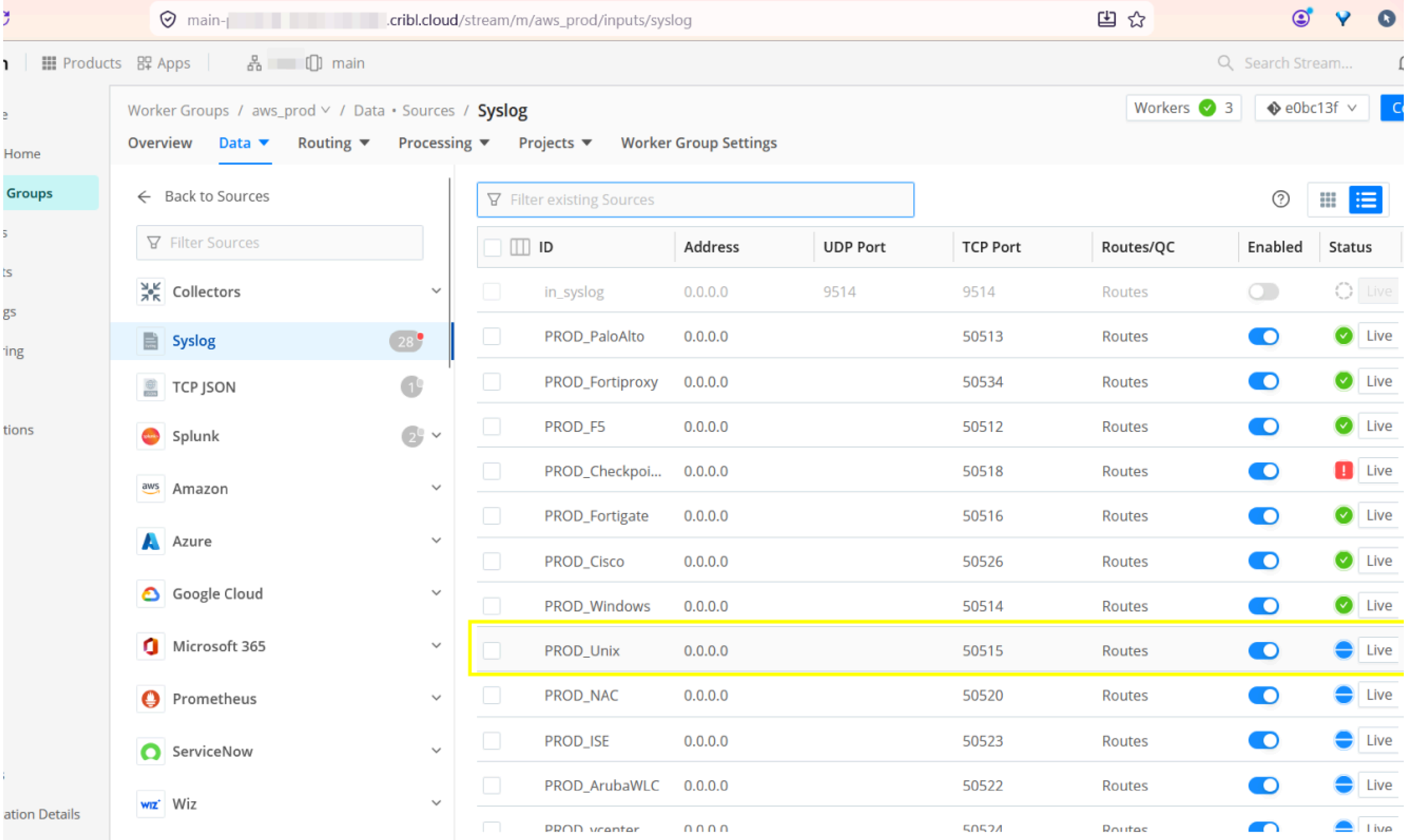
Syslog

Example Value | Schema

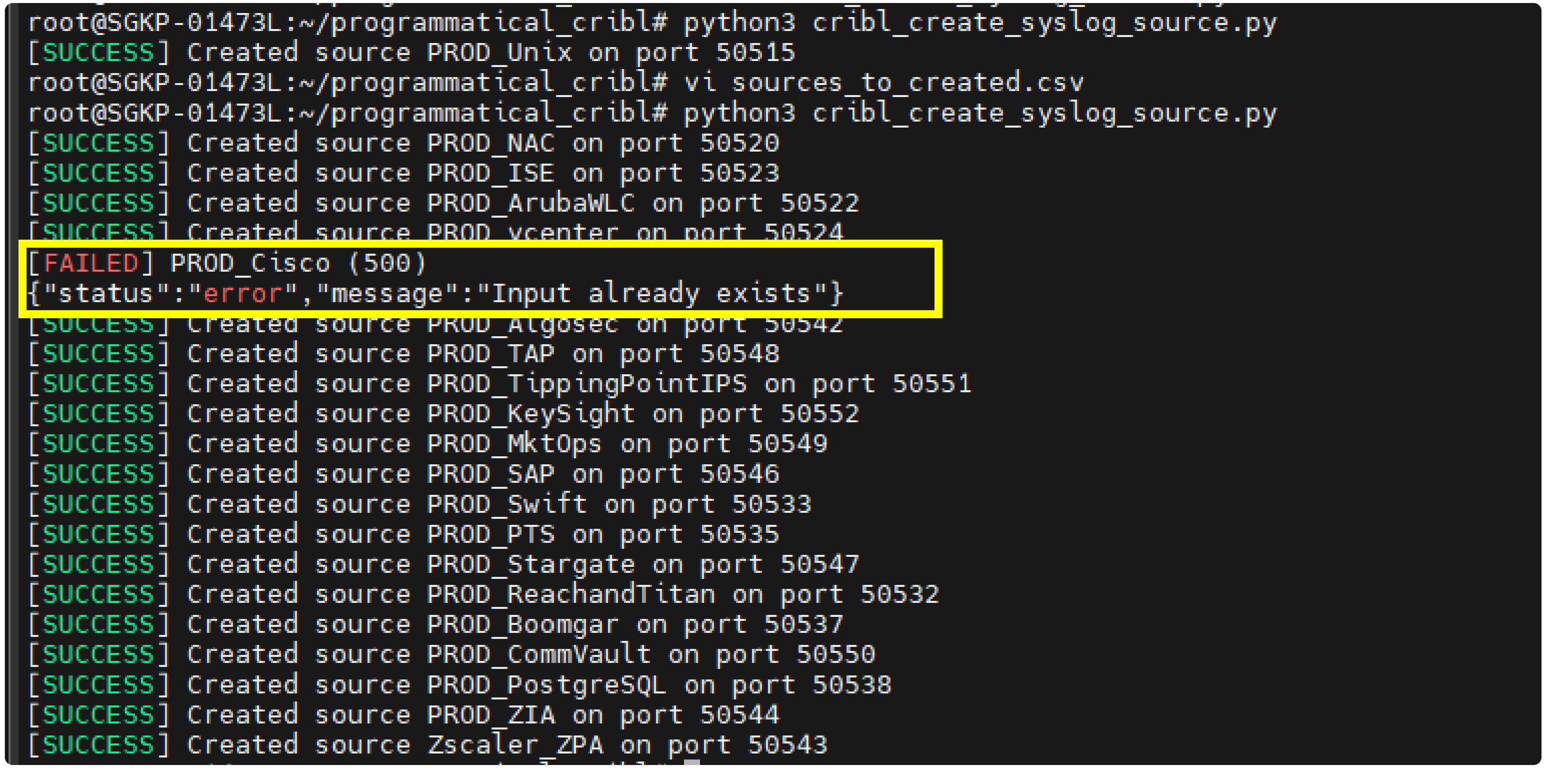
```
{
  "id": "syslog-source",
  "type": "syslog",
  "host": "0.0.0.0",
  "udpPort": 514,
  "sendToRoutes": true,
  "pqEnabled": false
}
```

After running the cURL command, we successfully created our source.

```
root@SGKP-01473L:~/programmatical_cribl# python3 cribl_create_syslog_source.py
[SUCCESS] Created source PROD_Unix on port 50515
```



Source definitions are maintained in a CSV file and processed in bulk to create the remaining 20 Cribl Sources in a single execution. It was observed that in cases of input ID naming conflicts, the creation fails for the affected entries, returning a “Failed” status along with the corresponding error message.



```
root@SGKP-01473L:~/programmatical_cribl# python3 cribl_create_syslog_source.py
[SUCCESS] Created source PROD_Unix on port 50515
root@SGKP-01473L:~/programmatical_cribl# vi sources_to_created.csv
root@SGKP-01473L:~/programmatical_cribl# python3 cribl_create_syslog_source.py
[SUCCESS] Created source PROD_NAC on port 50520
[SUCCESS] Created source PROD_ISE on port 50523
[SUCCESS] Created source PROD_ArubaWLC on port 50522
[SUCCESS] Created source PROD_vcenter on port 50524
[FAILED] PROD_Cisco (500)
{"status":"error","message":"Input already exists"}
[SUCCESS] Created source PROD_Algosec on port 50542
[SUCCESS] Created source PROD_TAP on port 50548
[SUCCESS] Created source PROD_TippingPointIPS on port 50551
[SUCCESS] Created source PROD_KeySight on port 50552
[SUCCESS] Created source PROD_MktOps on port 50549
[SUCCESS] Created source PROD_SAP on port 50546
[SUCCESS] Created source PROD_Swift on port 50533
[SUCCESS] Created source PROD_PTS on port 50535
[SUCCESS] Created source PROD_Stargate on port 50547
[SUCCESS] Created source PROD_ReachandTitan on port 50532
[SUCCESS] Created source PROD_Boomgar on port 50537
[SUCCESS] Created source PROD_CommVault on port 50550
[SUCCESS] Created source PROD_PostgreSQL on port 50538
[SUCCESS] Created source PROD_ZIA on port 50544
[SUCCESS] Created source Zscaler_ZPA on port 50543
```

Get/Display the specified source by ID

GET `/system/inputs/{id}` Get a Source

Get the specified Source.

Parameters

Name	Description
id * required `string` *(path)*	The `id` of the Source to get. id

After running the cURL command, we successfully view the created our source.

```
~/programmatical_cribl# python3 cribl_validate_syslog_source.py


 'PROD_Unix', 'description': 'Unix Datasources', 'type': 'syslog', 'host': '0.0.0.0', 'tcpPort': 50515, 'sendToRoutes': True, 'pqEnabled': False, 'disabled': False, 'streamta
'ipWhitelistRegex': '/.*/', 'timestampTimezone': 'local', 'singleMsgUdpPackets': False, 'enableProxyHeader': False, 'keepFieldsList': [], 'octetCounting': False, 'inferFramin
nting': True, 'allowNonStandardAppName': False, 'maxActiveCxn': 1000, 'socketIdleTimeout': 0, 'socketEndingMaxWait': 30, 'socketMaxLifespan': 0, 'enableLoadBalancing': False,
timestamp': 1780917672492, 'metrics': {'items': [], 'count': 0}}, 'notifications': []}], 'count': 1}
```

[Reference]

1. Cribl Python SDK (Control Plane)
   https://github.com/criblio/cribl_control_plane_sdk_python/tree/main

2. Cribl Python SDK: Configure Resources (Sources/Destinations/Routes/Pipelines)
   https://docs.cribl.io/cribl-as-code/sdk-configure-resources/